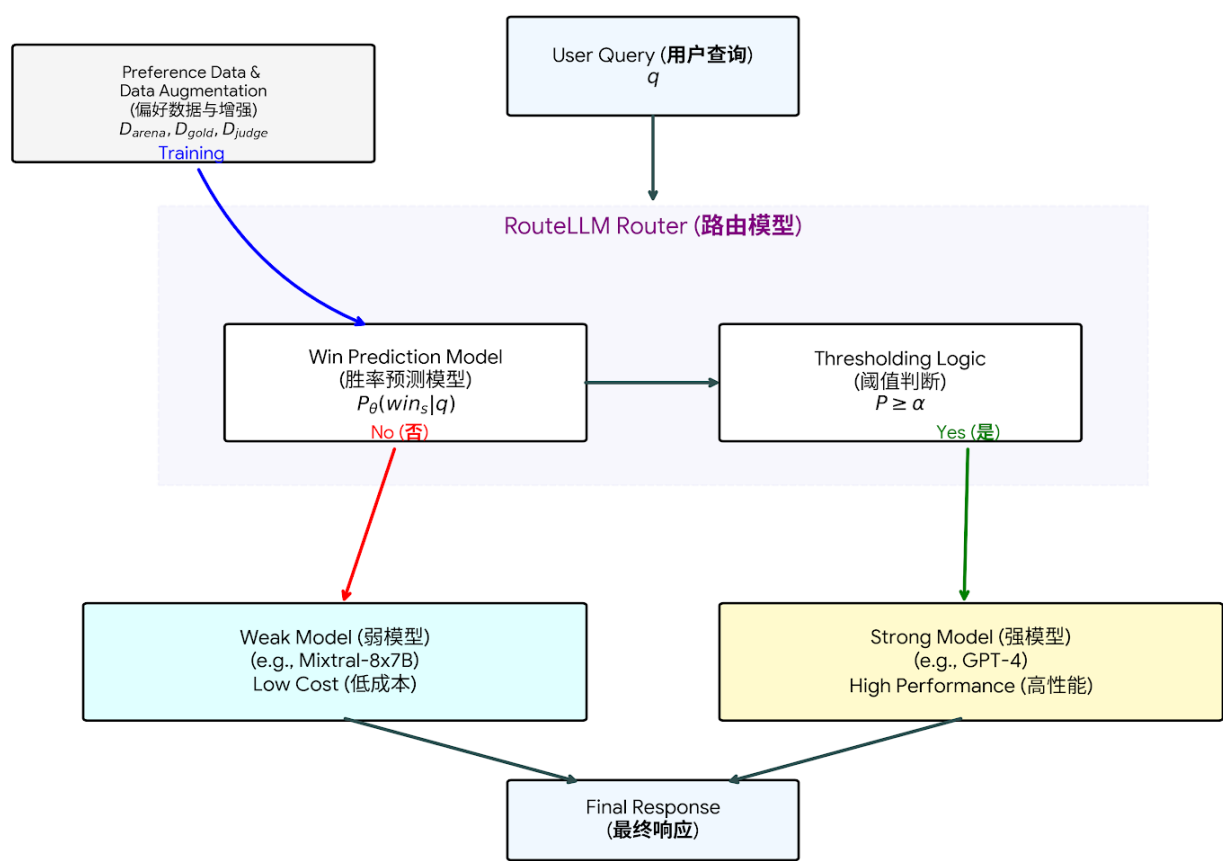


SLM-LLM Token-Level Collaborative-Decoding_Xinlin_Peng

1. RouteLLM (Query-Level Router)

[ICLR25] ROUTELLM: LEARNING TO ROUTE LLMS WITH PREFERENCE DATA
(<https://arxiv.org/abs/2406.18665>) 400+引用量

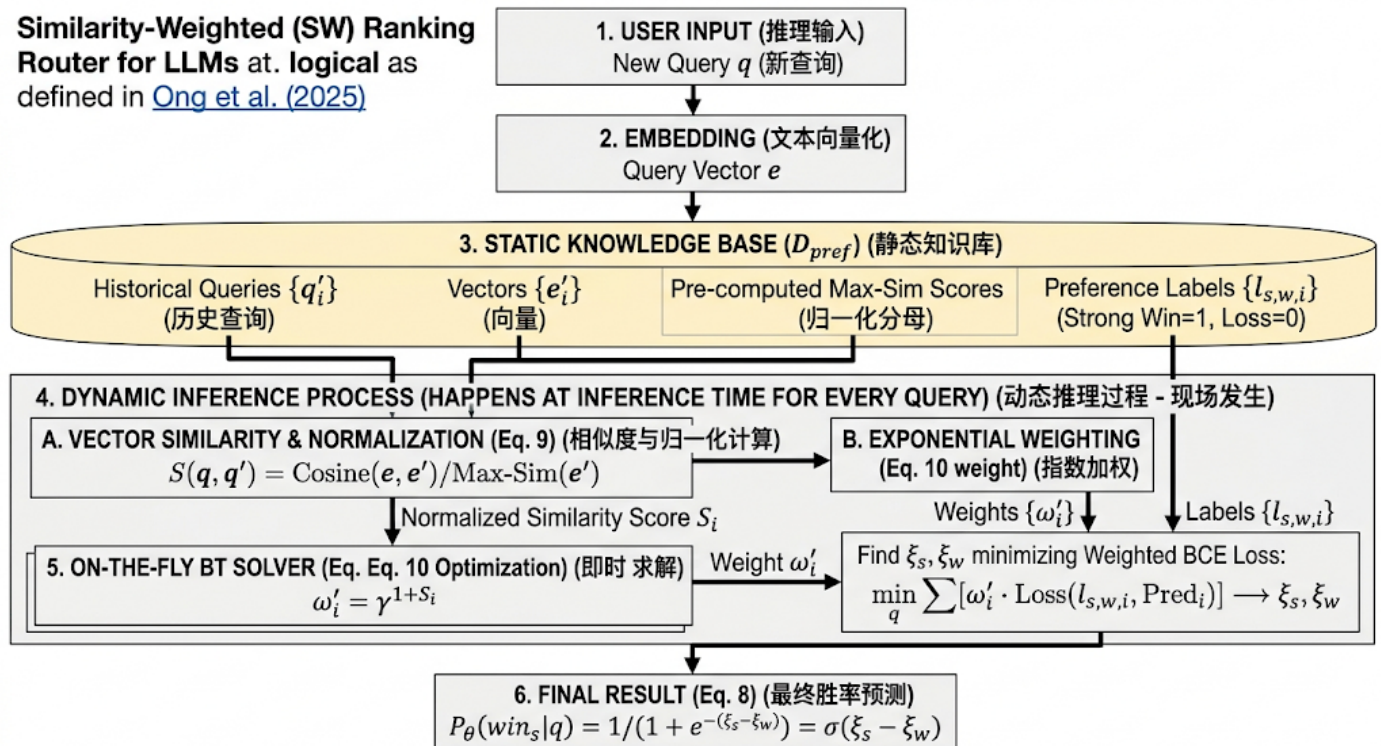


1. **输入与预测 (Input & Prediction)**: 用户查询 q 进入路由器。内部的**胜率预测模型** (如 Matrix Factorization 或 BERT) 计算强模型能够胜出的概率 $P_{\theta}(win_s|q)$ 。
2. **动态决策 (Decision Logic)**: 系统根据预设的**成本阈值** α 进行判断:
 - $P \geq \alpha$: 将任务交给 **强模型 (Strong LLM)**, 追求高质量结果。
 - $P < \alpha$: 将任务交给 **弱模型 (Weak LLM)**, 以节省成本。
3. **数据增强 (Data Augmentation)**: 通过 **Chatbot Arena** 的偏好数据进行训练, 并引入 **Gold Data** (带有标准答案的数据) 和 **LLM-judge** (如 GPT-4 自动标注) 来增强路由器的判别能力。

1.1 Query Router (Win Prediction Model)

1. Similarity-weighted (SW) ranking:

Similarity-Weighted (SW) Ranking Router for LLMs at. logical as defined in [Ong et al. \(2025\)](#)



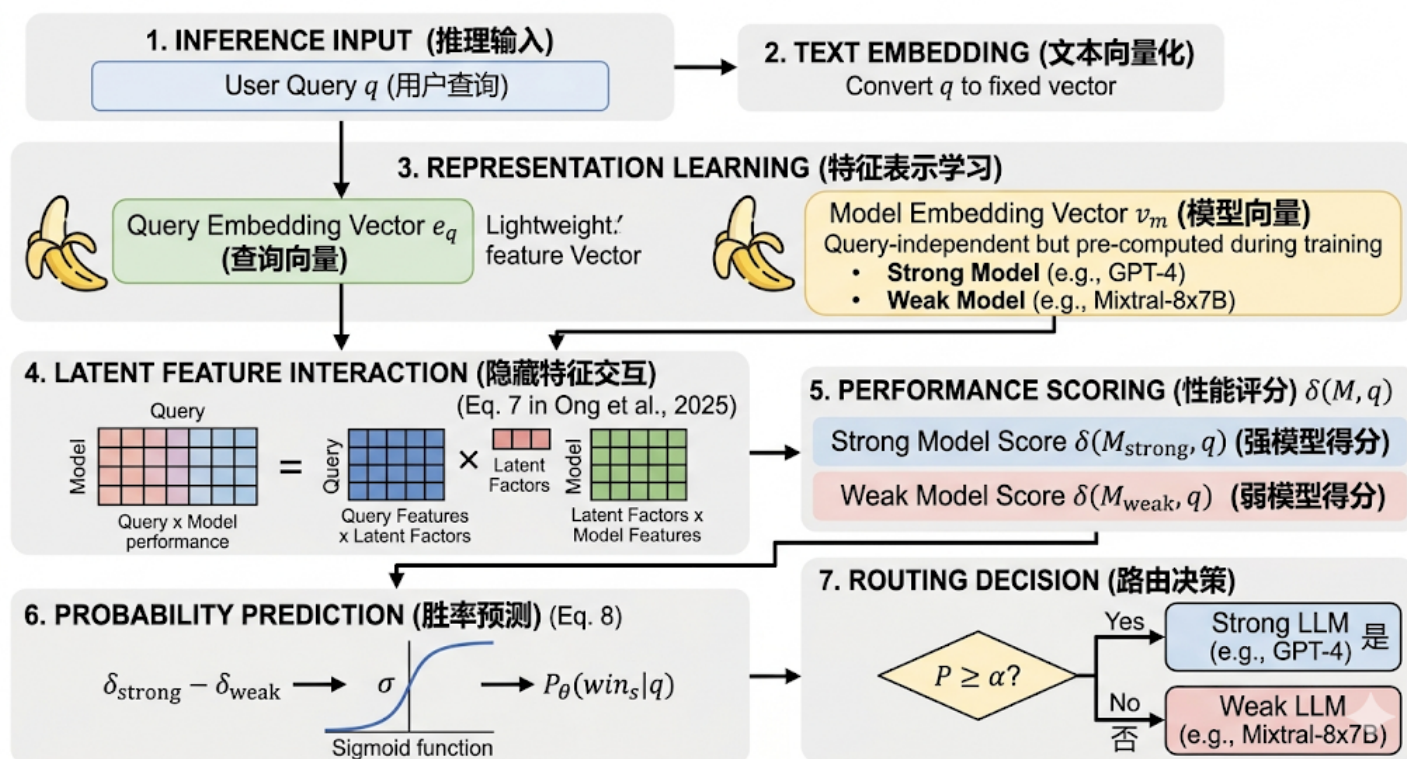
- 为什么需要归一化分数

想象你在参加一个**“寻找失散兄弟”**的活动：

- **原始分（分子）**：你和某人的外貌相似度是 80 分。
- **归一化因子（分母）**：那个人在他自己的家族里，和他长得最像的双胞胎兄弟相似度是 95 分。
- **最终得分 S** ： $80/95 = 0.84$ 。这说明在“长得像”这件事上，你已经达到了他双胞胎水平的 84%，这很有参考价值。
- **反例**：如果那个人是“大众脸”，跟路人相似度都有 99 分，那么 $80/99 = 0.80$ 。虽然分子一样，但因为他太大众了，你的参考价值反而下降了。

成本：不用离线训练，但是要扫描整个代码库，非常慢

2. Matrix factorization



- 学习性能评分函数：

$$\delta(M, q) = w_2^T (v_m \odot (W_1^T v_q + b))$$

成本：8GB显存的GPU上训练10个epoch

3. BERT classifier：作为文本分类任务

$$P_{\theta}(\text{win}_s|q) = \sigma(Wh_{\text{CLS}} + b),$$

成本：在2张L4 24GB GPU上训练2000步

4. Causal LLM classifier：引入一个LLM（Llama3 8B），微调让它输出使用不同模型的概率

成本：8张A100 80GB 训练2000步

1.2 数据增强

- 数据增强方法
 - a. 让大模型和小模型分别回答问题，看对错，对的模型判胜，都对判平，错的判负
 - b. 用LLM judge不同的回答，直接评判是否好（比如都答对的情况）
- 为什么需要数据增强
 - 模型组合数量极大：因为chatbot arena里面有很多不同模型的问答对
 - 覆盖领域不均衡：
 - Arena中的问题非常随机（比如“写个笑话”或“帮我润色邮件”）
 - 缺少专业性强的领域（高等数学，医学法律）

1.3 结果

Training data	Method	<i>CPT</i> (50%)	<i>CPT</i> (80%)	<i>APGR</i>	Improvement
	Random (95% CI)	49.03(±4)%	78.08(±3)%	0.500(±0.02)	(+0%)
$\mathcal{D}_{\text{arena}}$	BERT	78.09%	87.64%	0.391	(-21.8%)
	Causal LLM	28.82%	77.53%	0.573	(+14.6%)
	Matrix Factorization	25.32%	74.26%	0.580	(+16%)
	SW Ranking	37.85%	58.99%	0.610	(+22.1%)
$\mathcal{D}_{\text{arena}} + \mathcal{D}_{\text{judge}}$	BERT	19.58%	34.02%	0.751	(+50.2%)
	Causal LLM	31.50%	48.75%	0.679	(+35.8%)
	Matrix Factorization	13.40%	31.31%	0.802	(+60.4%)
	SW Ranking	23.21%	36.04%	0.759	(+51.8%)

Table 1: MT Bench results. Note that the MT Bench score at *CPT*(50%), 8.8, is 95% that of GPT-4’s score (9.3). Our routers exhibit strong performance on MT Bench when trained on $\mathcal{D}_{\text{arena}}$, with further improvement when the dataset is augmented with $\mathcal{D}_{\text{judge}}$, reducing costs by up to 75% as compared to the random router.

- 强模型是GPT-4，小模型是Mixtral
- CPT：为了恢复强弱模型之间 50% 或 80% 的性能差距，需要调用强模型（GPT-4）的比例。
- APGR：平均恢复性能差距，代表路由器在所有成本预算（通过调用alpha）下的整体表现。

由于我们可以通过调整路由器的“门槛”（阈值 α ）来控制成本，因此会得到一条**性能-成本曲线**。
APGR 就是这条曲线下的**面积**：

$$APGR(M_{R^\alpha}) = \int_0^1 PGR(M_{R^\alpha}) d(c(M_{R^\alpha}))$$

- Average Response Quality:

$$r(M_{R^\alpha}) = \frac{1}{|Q|} \sum_{q \in Q} s(M_{R^\alpha}(q)),$$

$s(\cdot)$ 代表单条回复的质量分数。

- the percentage of calls to strong models:

$$c(M_{R^\alpha}) = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}\{R^\alpha(q) = \mathcal{M}_{\text{strong}}\},$$

- PGR:

$$PGR(M_{R^\alpha}) = \frac{r(M_{R^\alpha}) - r(M_w)}{r(M_s) - r(M_w)}$$

- $r(M_w)$: 弱模型 (如 Mixtral) 的得分。
- $r(M_s)$: 强模型 (如 GPT-4) 的得分。
- $r(M_{R^\alpha})$: 路由器在设定阈值为 α 时的得分。
- **含义**: 如果 PGR 是 0.8，意味着路由器已经填补了弱模型与强模型之间 80% 的性能鸿沟。

1.4 泛化性

• 数据集泛化性

Training data	Method	<i>CPT</i> (50%)	<i>CPT</i> (80%)	<i>APGR</i>	Improvement
	Random (95% CI)	50.00(±2)%	80.08(±1)%	0.497(±0.01)	(+0%)
$\mathcal{D}_{\text{arena}}$	BERT	58.78%	83.84%	0.438	(-11.8%)
	Causal LLM	56.09%	83.56%	0.461	(-7.3%)
	Matrix Factorization	53.59%	85.24%	0.4746	(-4.5%)
	SW Ranking	54.43%	82.11%	0.4753	(-4.3%)
$\mathcal{D}_{\text{arena}} + \mathcal{D}_{\text{judge}}$	BERT	44.76%	79.09%	0.531	(+6.9%)
	Causal LLM	33.64%	63.26%	0.622	(+25.3%)
	Matrix Factorization	38.82%	72.62%	0.565	(+13.8%)
	SW Ranking	41.21%	72.20%	0.568	(+14.3%)

Table 3: 8-shot GSM8K results. Note that the GSM8K score at *CPT*(50%), 75, is 87% that of GPT-4’s score (86). Routers trained only on $\mathcal{D}_{\text{arena}}$ again perform poorly due to questions being out-of-distribution, but augmentation with $\mathcal{D}_{\text{judge}}$ substantially improves router performance.

结论：数据增强非常有必要

• 模型泛化性

Model Pair (M_s / M_w)	Method	<i>CPT</i> (50%)	<i>CPT</i> (80%)	<i>APGR</i>	Improvement
Claude 3 Opus / Claude 3 Sonnet	Random (95% CI)	49.89 (±3)%	72.27 (±4)%	0.493 (±0.033)	(+0%)
	BERT	34.85%	39.04%	0.682	(+38.3%)
	Causal LLM	28.12%	50.00%	0.656	(+33.1%)
	Matrix Factorization	31.86%	36.43%	0.762	(+54.6%)
	SW Ranking	23.27%	51.85%	0.772	(+56.6%)
Llama 3.1 70B / Llama 3.1 8B	Random (95% CI)	47.52(±3)%	76.26(±2)%	0.512 (±0.017)	(+0%)
	BERT	30.15%	38.91%	0.673	(+31.4%)
	Causal LLM	34.05%	45.96%	0.689	(+34.6%)
	Matrix Factorization	25.83%	37.30%	0.738	(+44.1%)
	SW Ranking	21.18%	29.39%	0.767	(+49.8%)

Table 4: MT Bench results for our routers when used to route between different model pairs. We use the exact same routers as before trained on $\mathcal{D}_{\text{arena}} + \mathcal{D}_{\text{judge}}$. Our routers generalize very well across different model pairs without any retraining.

结论：在GPT-4和Mixtral模型对上训练的router，可以直接迁移到Claude和Llama上

2. R2R (Token-Level)

- [Neurips2025] R2R: Efficiently Navigating Divergent Reasoning Paths with Small-Large Model Token Routing <https://arxiv.org/abs/2505.21600>

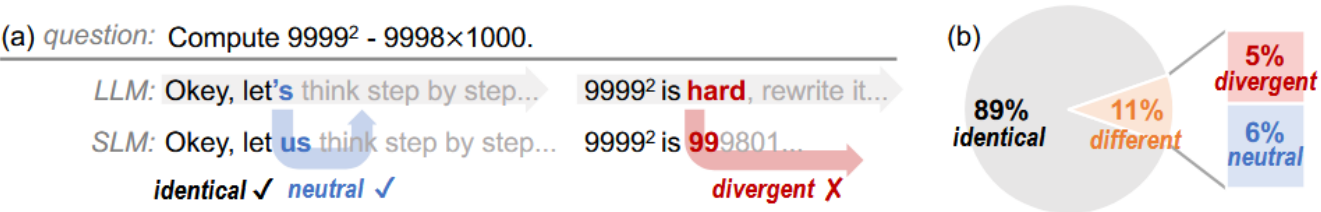


Figure 1: (a) Examples of R2R routing objective. Given a partial response as context, if SLM next-token prediction is not *identical* with LLM's, it is further categorized as *neutral* or *divergent* based on their effects on the reasoning path. (b) Distribution of *identical*, *neutral* and *divergent* labels in the R2R training set with 7.6M token labels.

将大小模型在同一位置的点分为：

- 1. **Identical**：大小模型完全相同
- 2. **Different**：大小模型不相同
 - a. **Divergent**：完全不同意思
 - b. **Neural**：同义词转换

2.1 Neural Router的训练数据生成



Figure 3: R2R data labeling pipeline. Given a query question, the LLM first generates a response to establish the desired reasoning path. The SLM then prefills this path to identify *identical* and *different* next-token predictions. For each *different* SLM token, the LLM continues generation from that point. Finally, a verifier model determines whether each difference leads to a *neutral* or *divergent* outcome, labeling the model preference as SLM or LLM, respectively.

- 1. **Response**：大模型先生成一段response
- 2. **SLM Prefill**：小模型一步一步prefill，找和大模型不同的位置（mismatch）
- 3. **Continuation**：让大模型沿着不同的路径续写
- 4. **Verify**：由一个超大的verifier进行评判，是同义还是不同义，进行标注

- 训练数据的细粒度统计

Table 1: Statistics of tokens difference and divergence across query types in the training dataset.

Type	#Query	#Token	#Different	Diff. Rate	#Divergent	Div. Rate
Math	587	2.9M	195.1K	6.8%	81.8K	2.8%
Code	698	3.2M	329.0K	10.3%	151.9K	4.7%
QA	735	1.4M	290.8K	20.2%	139.4K	9.7%
Summary	2094	7.6M	814.9K	10.8%	373.1K	4.9%

数据生成成本：训练数据的生成耗时非常长

论文中关于**训练时长**和**数据生成耗时**的详细数据主要分布在第 4.1 节、附录 A.3 和附录 B.3.1 中。根据作者提供的实验记录，整个流程从零开始到模型就绪大约需要 **2.5 到 3 天**（基于 8 张 NVIDIA A800-80GB GPU 的硬件配置）。

以下是具体的时间分布：

1. 数据生成阶段 (Data Labeling Pipeline)

这是最耗时的环节，论文在附录表 9 中列出了四阶段流水线的详细延迟：

- **总耗时**：56 小时（约 2.3 天），共计消耗 448 GPU 小时。
- **各阶段拆解**：
 - **LLM 响应生成 (Step 0)**：35 小时（占据了总时长的 62.5%）。
 - **SLM 预填 (Step 1)**：0.1 小时（非常高效）。
 - **LLM 延续生成 (Step 2)**：7 小时。
 - **验证与判定 (Step 3)**：14 小时。

2.2 Router和熵以及出现频率有关系

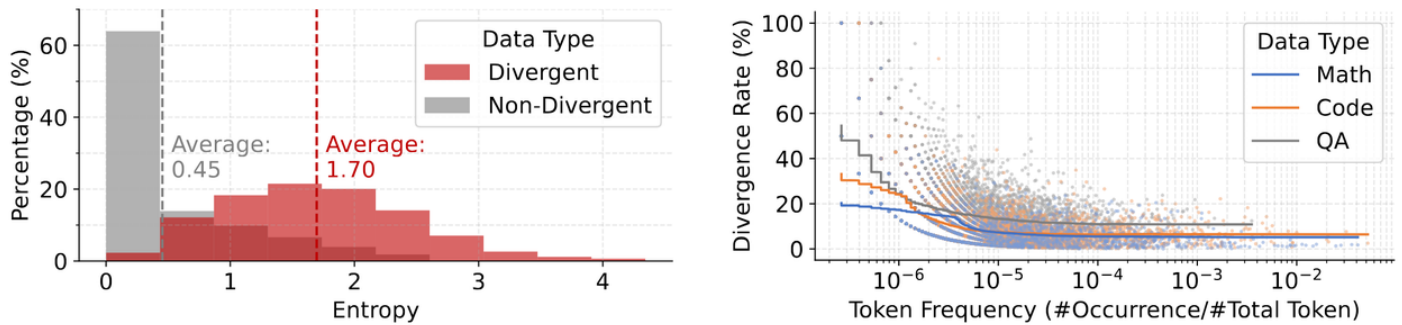


Figure 4: Oracle insights for router design. (a) SLM entropy distribution, clipped at 99th percentile for visualization clarity (b) Divergence rate and frequency of different tokens.

1. 高熵容易divergent
2. 以及出现频率低的容易divergent

2.3 Neural Router

网络结构: 6层56M的FFN

labels数量: 7.6M token-level routing labels

训练好之后，Router的**输入**:

1. SLM top100 Logits: (关注熵)
2. Token embedding (去注意出现频率低的token)
3. SLM Last-layer Hidden States (提供额外的语义上下文信息)

输出: 是否divergent的二分类概率 (阈值控制输出)

2.4 实验

- Performance

Table 2: Performance and efficiency comparison across benchmarks and methods. *Param.* denotes the average activated parameters per token in billions; *Cost* is the average output tokens (thousands) per query multiplied by average parameters (billions) .

Type	Method	AIME			LiveCodeBench			GPQA			Average		
		Acc.	Param.	Cost	Acc.	Param.	Cost	Acc.	Param.	Cost	Acc.	Param.	Cost
SLM	R1-1.5B	12%	1.5	42	9%	1.5	43	8%	1.5	42	10%	1.5	42
LLM	R1-32B	57%	32.0	487	45%	32.0	606	46%	32.0	519	50%	32	537
7B	R1-7B	32%	7.0	148	24%	7.0	168	29%	7.0	147	28%	7.0	154
	QR-SW	27%	7.2	168	16%	7.1	188	16%	7.1	179	20%	7.1	178
	QR-LLM	27%	7.1	170	13%	7.1	195	19%	7.0	172	20%	7.1	179
	QR-BERT	28%	7.1	160	15%	7.0	189	21%	7.0	169	21%	7.0	173
	QR-MF	27%	7.5	168	16%	7.1	190	16%	7.1	181	20%	7.2	180
14B	R1-14B	48%	14.0	239	38%	14.0	267	44%	14.0	197	43%	14.0	234
	QR-SW	37%	14.5	295	27%	14.0	333	25%	14.1	318	30%	14.2	315
	QR-LLM	45%	14.8	277	21%	14.1	356	28%	14.1	299	31%	14.3	311
	QR-BERT	37%	14.0	280	25%	14.0	342	29%	14.1	297	30%	14.0	306
	QR-MF	42%	14.7	284	25%	14.0	336	20%	14.2	338	29%	14.3	319
R2R	Ours	55%	5.5	101	39%	5.1	106	44%	6.3	101	46%	5.6	103

Speed and Position

Method	#Token(K)	Latency(s)	Speed(tok/s)
R1-1.5B	28.2 $\pm$ 10.5	199 $\pm$ 81	141.6
R1-14B	17.1 $\pm$ 12.3	328 $\pm$ 272	52.1
R1-32B	15.2 $\pm$ 12.4	498 $\pm$ 456	30.5
QR-SW	20.3 $\pm$ 13.5	336 $\pm$ 379	55.5
QR-LLM	18.7 $\pm$ 13.5	332 $\pm$ 334	56.1
QR-BERT	19.9 $\pm$ 13.2	350 $\pm$ 367	57.0
QR-MF	19.3 $\pm$ 13.3	347 $\pm$ 359	55.6
Eagle2	17.4 $\pm$ 13.1	244 $\pm$ 194	71.4
HASS	18.8 $\pm$ 12.9	256 $\pm$ 197	73.3
Ours	18.4 $\pm$13.5	218 $\pm$161	84.3

Table 3: Comparison of latency, output token length, and average speed across methods. Subscripts note the standard deviations across AIME.

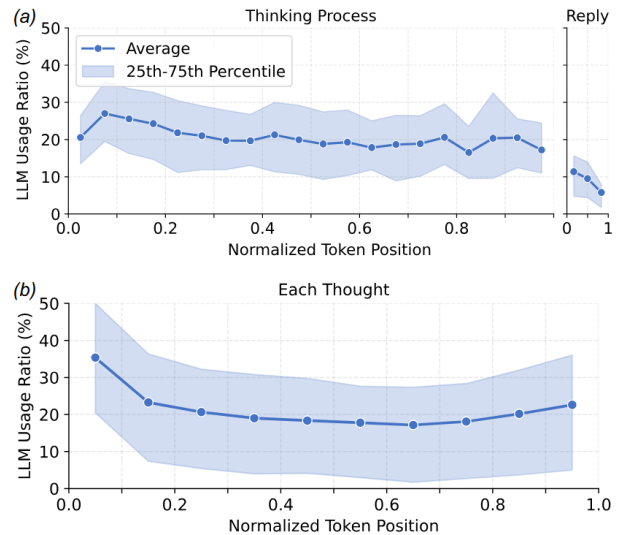


Figure 6: LLM usage rate at different positions, normalized by (a) thinking and reply process, (b) each thought.

2.5 泛化性的问题

数据集的泛化性:

Table 7: Performance of R2R on Arena-Hard for Dialog and MMLU-Redux-Philosophy.

Type	Model(s)	Dialog			Philosophy		
		Score	Param.	Cost	Acc.	Param.	Cost
LLM	R1-32B	5.0	32	65.9	79 %	32	32.2
SLM	R1-1.5B	0.2	1.5	14.3	38 %	1.5	8.7
Distill	R1-7B	0.3	7	35.0	57 %	7	20.6
Distill	R1-14B	2.2	14	56.5	77 %	14	18.5
R2R	Ours	2.8	6.1	40.9	81 %	6.7	8.3

泛化性确实比较好，我在mmlu chemistry/biology上测试r2r router的方法基本上也能达到LLM的效果

- 路由器的泛化性：

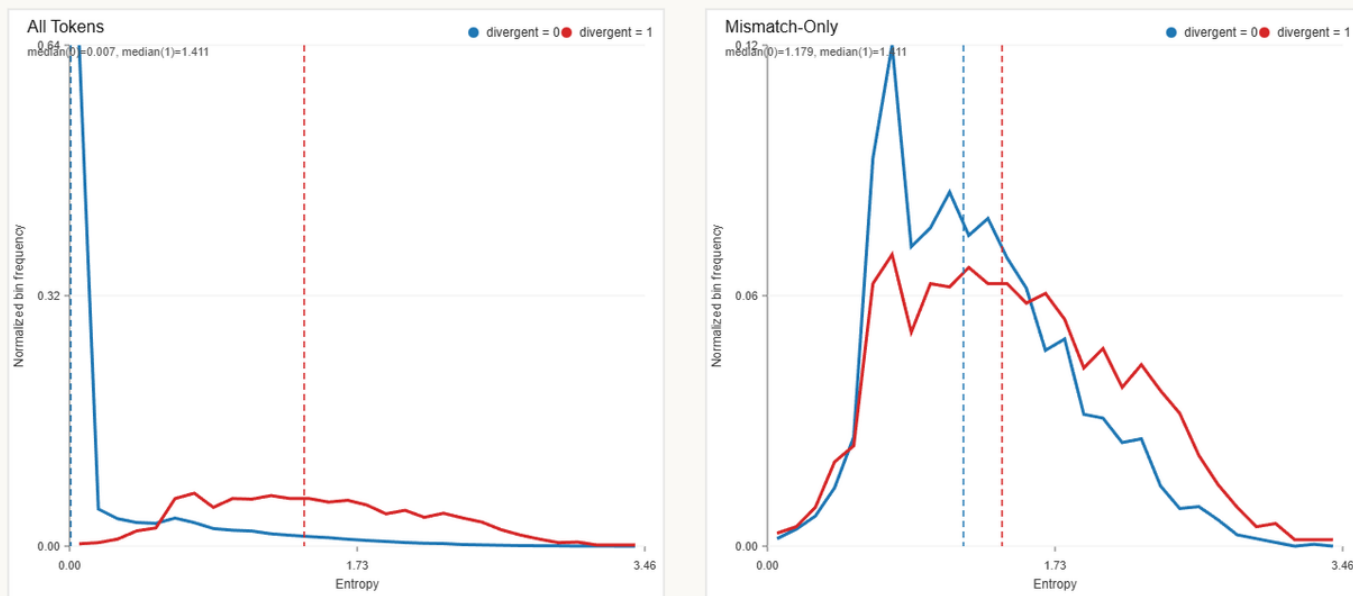
Table 8: Router generalizability on the AIME benchmark. *Param.* denotes the average activated parameters per token in billions; *Cost* is the average output tokens (thousands) per query multiplied by average parameters (billions) .

Type	Model(s)	Router Variant	Acc.	Param.	Cost
SLM	Qwen3-0.6B	-	12%	0.6	15
LLM	Qwen3-8B	-	67%	8.0	130
LLM	Qwen3-32B	-	75%	32.0	469
R2R	0.6B+8B	Trained on 0.6B+8B	65%	2.7	49
		Generalized from 0.6B+32B	53%	2.6	50
R2R	0.6B+32B	Trained on 0.6B+32B	67%	10.2	154
		Generalized from 0.6B+8B	70%	9.9	151

我怀疑泛化性本身就来源于，其实学到的都是高entropy/高散度的token

R2R Divergent vs Non-Divergent Entropy Distributions

Blue = non-divergent, Red = divergent. Dashed lines mark medians. The mismatch-only panel is the key one for testing whether entropy alone explains R2R labels.



2.6 Beyond Greedy decoding

会调整生成训练数据时的打标策略，mismatch的位置走k个分支，每个分支会用贪心采样的方式生成一段，会根据发散的概率判断当前位置是不是要label为divergent。

1. 需要重新调整生成数据的策略重新训练新router
2. 文章结尾提到了只在有限的一组数据（AIME）上取得了好结果

Table 18: Performance and efficiency comparison on AIME and methods with sampling. *Param.* denotes the average activated parameters per token in billions; *Cost* is the average output tokens (thousands) per query multiplied by average parameters (billions) .

Method	Acc.	Param. (B)	Cost (KB)
R1-1.5B	27%	1.5	24.3
R1-32B	61%	32.0	384.8
R1-14B	58%	14.0	170.5
QR-SW	39%	9.2	135.5
QR-LLM	38%	9.2	138.6
QR-BERT	40%	9.4	136.8
QR-MF	41%	9.1	130.5
R2R	58%	8.6	115.3

2.7 和Speculative Sampling的差距

不用回溯，立即判断